

COMP 3311

DATABASE MANAGEMENT

SYSTEMS

LECTURE 15 EXERCISES

MONGODB DBMS:

AGGREGATION FRAMEWORK

BOATRESERVATIONS MONGODB DATABASE

sailors collection

```
{sailorId: 10, sName: 'Emily', hkid: 'P936219(3)', age: 16},
{sailorId: 15, sName: 'Zoey', hkid: 'K091876(5)', rating: null, age: 17},
{sailorId: 22, sName: 'Dustin', hkid: 'A298456(6)', rating: 7, age: 45,
  reservations: [{boatId: 103, rDate: '08-SEP-25'},
                 {boatId: 104, rDate: '10-OCT-25'},
                 {boatId: 101, rDate: '10-OCT-25'},
                 {boatId: 102, rDate: '10-NOV-25'}]},
{sailorId: 29, sName: 'Brutus', hkid: 'D746239(0)', rating: 1, age: 33},
{sailorId: 31, sName: 'Lucy', hkid: 'P930281(5)', rating: 10, age: 55,
  reservations: [{boatId: 102, rDate: '10-OCT-25'},
                 {boatId: 103, rDate: '06-NOV-25'},
                 {boatId: 102, rDate: '12-NOV-25'}]},
{sailorId: 32, sName: 'Andy', hkid: 'M028391(4)', rating: 8, age: 25},
{sailorId: 58, sName: 'Rachael', hkid: 'A819204(3)', rating: 10, age: 17},
{sailorId: 64, sName: 'Horatio', hkid: 'F710943(2)', rating: 7, age: 35,
  reservations: [{boatId: 101, rDate: '05-SEP-25'},
                 {boatId: 102, rDate: '08-SEP-25'}]},
{sailorId: 71, sName: 'Zorba', hkid: 'P910562(4)', rating: 1, age: 16},
{sailorId: 74, sName: 'Horatio', hkid: 'T810463(1)', rating: 9, age: 35,
  reservations: [{boatId: 103, rDate: '08-SEP-25'}]},
{sailorId: 85, sName: 'Amy', hkid: 'B719037(8)', rating: 3, age: 25},
{sailorId: 95, sName: 'Bob', hkid: 'E012735(5)', rating: 3, age: 63},
{sailorId: 99, sName: 'Chris', hkid: 'H814243(7)', rating: 10, age: 30,
  reservations: [{boatId: 102, rDate: '08-AUG-25'}]}
```

boats collection

```
{boatId: 101, bName: 'Interlake', color: 'blue', bType: 'dinghy',
  reservations: [{sailorId: 64, rDate: '05-SEP-25'},
                 {sailorId: 22, rDate: '10-OCT-25'}]},
{boatId: 102, bName: 'Interlake', color: 'red', bType: 'sloop',
  reservations: [{sailorId: 64, rDate: '08-SEP-25'},
                 {sailorId: 31, rDate: '10-OCT-25'},
                 {sailorId: 22, rDate: '10-NOV-25'},
                 {sailorId: 31, rDate: '12-NOV-25'},
                 {sailorId: 99, rDate: '08-AUG-25'}]},
{boatId: 103, bName: 'Clipper', color: 'green', bType: 'dinghy',
  reservations: [{sailorId: 22, rDate: '08-SEP-25'},
                 {sailorId: 74, rDate: '08-SEP-25'},
                 {sailorId: 31, rDate: '06-NOV-25'}]},
{boatId: 104, bName: 'Marine', color: 'red', bType: 'catamaran',
  reservations: [{sailorId: 22, rDate: '10-OCT-25'}]},
{boatId: 105, bName: 'Serenity', color: 'cyan'}
```



EXERCISE 1

Find any three sailors among those that made the largest number of reservations. Include in the query result objects fields sailor name and number of reservations. Order the query result objects by number of reservations descending.

Query {sName: 'Dustin', numReservations: 4}
result: {sName: 'Lucy', numReservations : 3}
 {sName: 'Horatio', numReservations : 2}

```
db.sailors.aggregate([
  {$match: {reservations: {$exists: true}}}, // eliminate sailors with no reservations
  {$project: {_id: 0, sName: 1, numReservations: {$size: '$reservations'}}},
  {$sort: {numReservations: -1}},           // sort by numReservations descending
  {$limit: 3}                             // retain only the first three results
]);
```

Is the **\$match** stage necessary? Why?

An error is raised by the **\$size** operator if a sailor has no reservation.

What if no sailor has made a reservation? What is the query result?

Can you give another solution to address this problem?

EXERCISE 1 (CONTD)

Find any three sailors among those that made the largest number of reservations. Include in the query result objects fields sailor name and number of reservations. Order the query result objects by number of reservations descending.

```
db.sailors.aggregate([
  { $project: { _id: 0, sName: 1, numReservations: { $cond: { if: { $isArray: '$reservations' },
                                                            then: { $size: '$reservations' },
                                                            else: 0 } } } },
  { $sort: { numReservations: -1 } }, // sort by numReservations descending
  { $limit: 3 }                     // retain only the first three results
]);
```

Can you give yet another solution?

EXERCISE 1 (CONTD)

Find any three sailors among those that made the largest number of reservations. Include in the query result objects fields sailor name and number of reservations. Order the query result objects by number of reservations descending.

```
db.sailors.aggregate([
  {$project: {_id: 0, sName: 1, numReservations: {$size: {$ifNull: ['$reservations', [ ]]} }},
  {$sort: {numReservations: -1}}, // sort by numReservations descending
  {$limit: 3}                    // retain only the first three results
]);
```

Can **find** be used for this query? **No!** Why?

Cannot sort on **numReservations**.
Therefore, cannot get sailors who made
the largest number of reservations.

EXERCISE 2

Find the average sailor age for each rating.
Include in the query result objects fields rating and its average age.
Order the query result objects by rating descending.

Query {rating: 10, avgAge: 34} {rating: 9, avgAge: 35} {rating: 8, avgAge: 25} {rating: 7, avgAge: 40}
result: {rating: 3, avgAge: 44} {rating: 1, avgAge: 24.5} {rating: null, avgAge: 16.5}

```
db.sailors.aggregate([
  {$group: {
    _id: '$rating',           // group by rating
    avgAge: {$avg: '$age'}    // calculate the average age for the rating
  }},
  {$sort: {'_id': -1}},      // sort average ratings ascending
  {$project: {_id: 0, rating: '$_id', avgAge: '$avgAge'}}
]);
```

EXERCISE 3

Find the sailors with the highest rating.
Include in the query result objects fields sailor id, name, rating and age.

Query {sailorId: 31, name: 'Lucy', rating: 10, age: 55} {sailorId: 58, name: 'Rachael', rating: 10, age: 17}
result: {sailorId: 99, name: 'Chris', rating: 10, age: 30}

```
db.sailors.aggregate([
  { $group: {
    _id: '$rating',
    sailors: { $push: { sailorId: '$sailorId',
                        sName: '$sName',
                        age: '$age' } }
  } },
  { $sort: { _id: -1 } },
  ...
])
```

// group by rating
// add sailors with
// this rating to the
// array field sailors
// order by rating descending

Query result after \$group and \$sort stages:

```
{_id: 10, sailors: [{sailorId: 31, name: 'Lucy', age: 55}, {sailorId: 58, name: 'Rachael', age: 35}, {sailorId: 99, name: 'Chris', age: 30}]}
{_id: 9, sailors: [{sailorId: 74, name: 'Horatio', age: 35}]}
{_id: 8, sailors: [{sailorId: 32, name: 'Andy', age: 25}]}
{_id: 7, sailors: [{sailorId: 22, name: 'Dustin', age: 45}, {sailorId: 64, name: 'Horatio', age: 35}]}
{_id: 3, sailors: [{sailorId: 85, name: 'Amy', age: 25}, {sailorId: 95, name: 'Bob', age: 63}]}
```



EXERCISE 3 (CONT'D)

Find the sailors with the highest rating.
Include in the query result objects fields sailor id, name, rating and age.

```
db.sailors.aggregate([
  { $group: {
    _id: '$rating',
    sailors: { $push: { sailorId: '$sailorId',
                       sName: '$sName',
                       age: '$age' } }
  } },
  { $sort: { _id: -1 } },
  { $limit: 1 },
  ...
])
```

// group by rating
// add sailors with
// this rating to the
// array sailors

// order by rating descending
// retain only the highest rating

Query result after \$limit stage:

```
{_id: 10, sailors: [{sailorId: 31, name: 'Lucy', age: 55}, {sailorId: 58, name: 'Rachael', age: 35}, {sailorId: 99, name: 'Chris', age: 30}]}
```

How to get the query result into the required format? **Unwind** sailors array.

EXERCISE 3 (CONTD)

Find the sailors with the highest rating.
Include in the query result objects fields sailor id, name, rating and age.

```
db.sailors.aggregate([
  { $group: {
    _id: '$rating',
    sailors: { $push: { sailorId: '$sailorId',
                        sName: '$sName',
                        age: '$age' } }
  } },
  { $sort: { _id: -1 } },
  { $limit: 1 },
  { $unwind: { path: '$sailors' } },
  { $project: { _id: 0, sailorId: '$sailors.sailorId', sName: '$sailors.sName',
               rating: '$_id', age: '$sailors.age' } }
]);
```

// group by rating
// add sailors with
// this rating to the
// array sailors

// order by rating descending
// retain only the highest rating
// for formatting the result

EXERCISE 4

Find sailors who have reserved a red boat.
Include in the query result objects field sailor name.
Order the query result objects by name ascending.

Query result: {sName: 'Chris'} {sName: 'Dustin'} {sName: 'Horatio'} {sName: 'Lucy'}

```
db.sailors.aggregate([
  {$lookup: {
    from: 'boats',
    localField: 'reservations.boatId',
    foreignField: 'boatId',
    as: 'boatReservations'
  }},
  ...
])
```

EXERCISE 4 (CONT'D)

Query result after \$lookup stage:

```
{sailorId: 99, sName: 'Chris', hkid: 'H814243(7)', rating: 10, age: 30, reservations: [{boatId: 102, rDate: '08-AUG-25'}],
  boatReservations: [{boatId: 102, bName: 'Interlake', color: 'red', type: 'sloop', reservations: [{sailorId: 64, rDate: '08-SEP-25'},
                                                                                               {sailorId: 31, rDate: '10-OCT-25'},
                                                                                               {sailorId: 22, rDate: '10-NOV-25'},
                                                                                               {sailorId: 31, rDate: '12-NOV-25'},
                                                                                               {sailorId: 99, rDate: '08-AUG-25'}]}}]}

{sailorId: 22, sName: 'Dustin', hkid: 'A298456(6)', rating: 7, age: 45, reservations: [{boatId: 103, rDate: '08-SEP-25'},
                                                                                     {boatId: 104, rDate: '10-OCT-25'},
                                                                                     {boatId: 101, rDate: '10-OCT-25'},
                                                                                     {boatId: 102, rDate: '10-NOV-25'}],
  boatReservations: [{boatId: 103, bName: 'Clipper', color: 'green', type: 'dinghy', reservations: [{sailorId: 22, rDate: '08-SEP-25'},
                                                                                               {sailorId: 74, rDate: '08-SEP-25'},
                                                                                               {sailorId: 31, rDate: '10-NOV-25'}]},
                     {boatId: 104, bName: 'Marine', color: 'red', type: 'catamaran', reservations: [{sailorId: 22, rDate: '07-OCT-25'}]},
                     {boatId: 101, bName: 'Interlake', color: 'blue', type: 'dinghy', reservations: [{sailorId: 64, rDate: '05-SEP-25'},
                                                                                               {sailorId: 22, rDate: '10-OCT-25'}]},
                     {boatId: 102, bName: 'Interlake', color: 'red', type: 'sloop', reservations: [{sailorId: 64, rDate: '08-SEP-25'},
                                                                                               {sailorId: 31, rDate: '10-OCT-25'},
                                                                                               {sailorId: 22, rDate: '10-NOV-25'},
                                                                                               {sailorId: 31, rDate: '12-NOV-25'},
                                                                                               {sailorId: 99, rDate: '08-AUG-25'}]}}]}

{sailorId: 15, sName: 'Zoey', hkid: 'K091876(5)', rating: null, age: 17,
  boatReservations: [ ]}
```

⋮



EXERCISE 4 (CONT'D)

Find sailors who have reserved a red boat.
Include in the query result objects field sailor name.
Order the query result objects by name ascending.

Query result: {sName: 'Chris'} {sName: 'Dustin'} {sName: 'Horatio'} {sName: 'Lucy'}

```
db.sailors.aggregate([
  {$lookup: {
    from: 'boats',
    localField: 'reservations.boatId',
    foreignField: 'boatId',
    as: 'boatReservations'
  }},
  {$match: {'boatReservations.color': 'red'}},
  {$project: {_id: 0, sName: 1}},
  {$sort: {sName: 1}}
]);
```

Is it necessary to unwind the boatReservations array before the **\$match** stage?

No, since only at most one match with a red boat is required to be in the query result.

EXERCISE 5

Find sailors for which the average rating of their age is equal to the minimum average rating of all ages.

Include in the query result object field minimum average rating and array field ages.
Order the ages array field by age ascending.

Query result: {minAvgRating: 1, ages: [16, 33]}

Is this a
correct
solution?

No. Why?

```
db.sailors.aggregate([
  { $match: { rating: { $ne: null } } }, // remove null ratings
  { $group: {
    _id: '$age', // group by age
    avgRating: { $avg: '$rating' } // calculate the average rating for the age
  } },
  { $sort: { avgRating: 1 } }, // sort average ratings ascending
  { $limit: 1 }, // retain only the minimum average rating
  { $project: { _id: 0, minAvgRating: '$avgRating', ages: '$_id' } }
]);
```

Query result: {minAvgRating: 1, ages: 16}

What's the problem?

Need to group again by average rating to get
all ages that have the minimum average rating.

EXERCISE 5 (CONTD)

Find sailors for which the average rating of their age is equal to the minimum average rating of all ages.

Include in the query result object field minimum average rating and array field ages.
Order the ages array field by age ascending.

Query result: {minAvgRating: 1, ages: [16, 33]}

```
db.sailors.aggregate([
  {$match: {rating: {$ne: null}}}, // remove null ratings
  {$group: {
    _id: '$age', // group by age
    avgRating: {$avg: '$rating'} // calculate the average rating for the age
  }},
  {$group: { // group again by average rating
    _id: '$avgRating', // to get all sailors with this rating
    ages: {$push: '$_id'} // collect ages for this average rating in an array
  }},
  {$sort: {'_id': 1}}, // sort average ratings ascending
  {$limit: 1}, // retain only the minimum average rating
  {$project: {_id: 0, minAvgRating: '$_id', ages: {$sortBy: {input: '$ages', sortBy: 1}}}}
]);
```

EXERCISE 6

Find the number of reservations made for each red boat.
Include in the query result objects fields boat id, boat name and
number of reservations.
Order the query result objects by number of reservations descending.

Query result: {boatId: 102, bName: 'Interlake', numReservations: 5} {boatId: 104, bName: 'Marine', numReservations: 1}

Is this a
correct
solution?

Partially!
Why?

It gets the
correct
result,
but...

```
db.boats.aggregate([
  {$match: {color: 'red'}},
  {$project: {_id: 0, boatId: 1, bName: 1,
    numReservations: {$size: '$reservations'}}},
  {$sort: {numReservations: -1}}
]);
```

What if there are red boats with no reservations?

This query raises the error:
The argument to \$size must be an array,
but was of type: missing.

How to fix this?

EXERCISE 6 (CONT'D)

Find the number of reservations made for each red boat.
Include in the query result objects fields boat id, boat name and
number of reservations.
Order the query result objects by number of reservations descending.

Query result: {boatId: 102, bName: 'Interlake', numReservations: 5} {boatId: 104, bName: 'Marine', numReservations: 1}

```
db.boats.aggregate([
  {$match: {color: 'red'}},
  {$project: {_id: 0, boatId: 1, bName: 1,
    numReservations: {$size: {$ifNull: ['$reservations', [ ]]}},
    {$sort: {numReservations: -1}}
  ]});
```

Can **find** be used for this query?

Yes, but then cannot sort on number of reservations made.

Can we use the **sailors** collection for this query? **Yes**, but ...

EXERCISE 6 (CONT'D)

Start from sailors collection

```
db.sailors.aggregate([
  {$unwind: {path: '$reservations'}},           // unwind the reservations array
  {$group: {
    _id: '$reservations.boatId',                  // count the number of
    numReservations: {$count: { }}              // reservations for each boat
  }},
  {$lookup: {                                     // get matching boat information
    from: 'boats',
    localField: '_id',
    foreignField: 'boatId',
    as: 'boatDetails'
  }},
  {$match: {'boatDetails.color': {$eq: 'red'}}}, // retain only the red boats
  {$project: {_id: 0, boatId: {$first: '$boatDetails.boatId'},
    bName: {$first: '$boatDetails.bName'}, numReservations: '$numReservations'},
  {$sort: {numReservations: -1}}
]);
```

EXERCISE 7

Find the ratings that have at least two adult (i.e., $\text{age} \geq 18$) sailors with the rating. Include in the query result objects fields rating, number of sailors with the rating and the age of the youngest adult sailor with the rating. Order the query result objects by rating ascending.

Query result: {rating: 3, numSailors: 2, minAge: 25} {rating: 7, numSailors: 2, minAge: 35} {rating: 10, numSailors: 3, minAge: 30}

Is this a
correct
solution?

No!
Why?

```
db.sailors.aggregate([
  { $match: { age: { $gte: 18 } } },           // eliminate sailors younger than 18
  { $group: {
    _id: '$rating',                           // group by rating
    numSailors: { $count: { } },              // count the number of sailors with this rating
    minAge: { $min: '$age' }                 // retain the minimum age for this rating
  } },
  { $match: { numSailors: { $gte: 2 } } },      // retain ratings with two or more adult sailors
  { $project: { _id: 0, rating: '$_id', numSailors: '$numSailors', minAge: '$minAge' } },
  { $sort: { rating: 1 } }
]);
```

Query result: {rating: 3, numSailors: 2, minAge: 25}
{rating: 7, numSailors: 2, minAge: 35}
{rating: 10, numSailors: 2, minAge: 30}

The count is wrong for rating 10.
What's the problem?



EXERCISE 7 (CONTD)

How about this solution?
Is it correct?

No!
Why?

```
db.sailors.aggregate([
  { $group: {
    _id: '$rating',                // group by rating
    numSailors: { $count: {} },    // count all sailors with this rating
    // count the number of adult sailors with this rating
    numAdultSailors: { $sum: { $cond: { if: { $gte: ['$age', 18] },
                                     then: 1,
                                     else: '$$REMOVE' } } },
    minAge: { $min: '$age' }      // retain the minimum age for this rating
  } },
  { $match: { numAdultSailors: { $gte: 2 } } }, // retain ratings with two or more adult sailors
  { $project: { _id: 0, rating: '$_id', numSailors: '$numSailors', minAge: '$minAge' } },
  { $sort: { rating: 1 } }
]);
```

Query {rating: 3, numSailors: 2, minAge: 25} {rating: 7, numSailors: 2, minAge: 35}
result: {rating: 10, numSailors: 3, minAge: 17}

The minimum age is wrong for rating 10.
What's the problem?

Should retain the minimum age only if it is 18 or greater.

EXERCISE 7 (CONTD)

```
db.sailors.aggregate([
  {$group: {
    _id: '$rating',                // group by rating
    numSailors: {$count: {}},      // count the total number of sailors with this rating
    // count the number of adult sailors with this rating
    numAdultSailors: {$sum: {$cond: {if: {$gte: ['$age', 18]},
                                   then: 1,
                                   else: '$$REMOVE'}}},
    // retain the age of the youngest adult sailor
    minAge: {$min: {$cond: {if: {$gte: ['$age', 18]},
                              then: '$age',
                              else: '$$REMOVE'}}}
  }},
  {$match: {numAdultSailors: {$gte: 2}}}, // retain ratings with two or more adult sailors
  {$project: {_id: 0, rating: '$_id', numSailors: '$numSailors', minAge: '$minAge'}},
  {$sort: {rating: 1}}
]);
```